

Session Overview

This session covers 1 hands-on project for **10-12 yrs: Capacitive touch sensor**. Total planned time is 120 minutes. Each project below includes materials, step-by-step instructions, full code or prompts, common mistakes, discussion questions, and extension challenges.

Schedule

ACTIVITY	SUGGESTED TIME
Welcome & setup	10 min
Capacitive touch sensor	102 min
Wrap-up & reflection	8 min

Capacitive touch sensor

Beginner · 10-12 yrs · ~60 min

Materials

- 1 x Arduino Uno board (Nerokas, nerokas.co.ke)
- 1 x USB cable (Type A to Type B) for programming the Arduino (Nerokas, nerokas.co.ke)
- 1 x TTP223 capacitive touch sensor module (Pixel Electronics, pixelelectronics.co.ke)
- 1 x LED (5mm, any colour, e.g. red) (Nerokas, nerokas.co.ke)
- 1 x 220 ohm resistor (current-limiting resistor for the LED) (Nerokas, nerokas.co.ke)
- 1 x Half-size or full-size breadboard (Pixel Electronics, pixelelectronics.co.ke)

- 6 x Male-to-male jumper wires (Nerokas, nerokas.co.ke)
- 1 x USB power bank or laptop (for powering the Arduino during the lesson) (not supplied by vendor — use existing classroom equipment)

How It Works

A capacitive touch sensor can tell when a human finger is nearby, without any physical button being pressed down. Our bodies naturally hold a tiny electrical charge. The touch sensor module has a small metal pad that is sensitive to this charge. When your finger gets close to or touches the pad, it changes the electrical field around the pad very slightly. The chip on the module (called a TTP223) notices this change and immediately sends out a clear, easy-to-read signal: either HIGH (which means "yes, someone is touching me") or LOW (which means "no touch detected").

Think of it like a doorbell that rings not because you pressed a physical button, but because you got close enough for it to notice you were there. The Arduino constantly "listens" to this signal on one of its digital pins. In our project, every time the sensor changes from "not touched" to "touched," the Arduino code flips the state of an LED — if the LED was off, it turns on; if it was on, it turns off. This is called "toggling," and it is different from just holding a button down to keep a light on. To make toggling work reliably, the code needs to notice the exact moment the touch begins, not repeat the toggle over and over while your finger stays on the pad. This is done by comparing the sensor's current reading to what it was a moment ago — a technique called edge detection.

Step-by-Step Instructions

1. Introduce the TTP223 touch sensor module to students. Point out its three pins: VCC (power), GND (ground), and OUT (output/signal). Explain that it also has a small touch pad marked on the board.
2. Place the LED on the breadboard, making sure the two legs are in different rows. Identify the longer leg (positive/anode) and the shorter leg (negative/cathode).
3. Connect the 220 ohm resistor in series with the LED: one end of the resistor to the LED's longer leg's row, and the other end of the resistor to an empty row that will connect to an Arduino digital pin.

4. Connect the LED's shorter leg (cathode) to the breadboard's ground rail.
5. Run a jumper wire from the breadboard's ground rail to the Arduino's GND pin.
6. Run a jumper wire from the row connected to the resistor to Arduino digital pin 8 (this will control the LED).
7. Connect the touch sensor module's VCC pin to the Arduino's 5V pin using a jumper wire.
8. Connect the touch sensor module's GND pin to the Arduino's GND pin (or to the breadboard ground rail, which is already connected to GND).
9. Connect the touch sensor module's OUT pin to Arduino digital pin 2 using a jumper wire.
10. Double-check all connections before powering on: VCC to 5V, GND to GND, OUT to pin 2, LED circuit to pin 8 and GND through the resistor.
11. Connect the Arduino to the computer using the USB cable. Open the Arduino IDE and select the correct board and port from the Tools menu.
12. Copy the full code (below) into a new sketch, then click Upload. Wait for the "Done uploading" message.
13. Ask students to touch the sensor pad once and observe: the LED should turn on. Ask them to touch it again and observe the LED turning off.
14. Discuss what they notice: does the LED stay on only while the finger is touching, or does it stay on after they let go? (It should stay on/off — that's the toggle behaviour.)
15. Encourage students to experiment with touching quickly versus slowly and note whether the toggle still works reliably each time.

Full Code

```
// Capacitive Touch Sensor - LED Toggle Project
// The LED changes state (on/off) each time the touch sensor is touched

const int touchPin = 2;    // Digital pin connected to the touch sensor's OUT pin
const int ledPin = 8;      // Digital pin connected to the LED (through the resistor)

bool ledState = false;     // Tracks whether the LED is currently on or off
int touchState = 0;        // Stores the current reading from the touch sensor
int lastTouchState = 0;    // Stores the previous reading, used to detect a new touch

void setup() {
  pinMode(touchPin, INPUT); // Set the touch sensor pin as an input
  pinMode(ledPin, OUTPUT);  // Set the LED pin as an output
  digitalWrite(ledPin, LOW); // Make sure the LED starts off
```

```

Serial.begin(9600);          // Start serial communication for debugging
}

void loop() {
  touchState = digitalRead(touchPin); // Read the current state of the touch sensor (HIGH or LOW)

  // Check if the sensor has just changed from not-touched (LOW) to touched (HIGH)
  // This "edge detection" stops the LED from flickering rapidly while the finger stays on the pad
  if (touchState == HIGH && lastTouchState == LOW) {
    ledState = !ledState;          // Flip the LED state: true becomes false, false becomes true
    digitalWrite(ledPin, ledState); // Apply the new state to the LED pin
    Serial.println(ledState ? "LED ON" : "LED OFF"); // Print the new state to the Serial Monitor for debugging

    delay(50); // Short delay to help avoid false re-triggers right after a touch (basic debounce)
  }

  lastTouchState = touchState; // Save the current reading so we can compare it next time through the loop
}

```

Common Mistakes & Fixes

SYMPTOM	LIKELY CAUSE	FIX
LED never turns on, no matter how the sensor is touched	OUT pin of the sensor connected to the wrong Arduino pin, or code using a different pin number than wired	Check that the OUT pin is on digital pin 2 and that <code>touchPin</code> in the code matches the wiring
LED flickers rapidly or toggles multiple times with a single touch	No debounce, or edge detection not implemented, so the fast loop catches "touched" state many times	Confirm the code compares <code>touchState</code> to <code>lastTouchState</code> and includes the short delay after a toggle
LED is always on, even without touching the sensor	LED wired directly to 5V instead of a digital pin, or resistor missing causing a short/bright always-on LED	Check the LED's longer leg connects through the 220 ohm resistor to pin 8, not directly to 5V
Sensor seems to trigger on its own, or the LED toggles without anyone touching it	Loose jumper wires causing noise, or hand/nearby metal objects close to the touch pad	Reseat all jumper wires firmly, keep hands and metal objects away from the pad when not testing
Nothing happens and Serial Monitor shows nothing	Wrong baud rate selected in Serial Monitor, or wrong board/port selected in Arduino IDE	Set Serial Monitor baud rate to 9600 and confirm correct board/port under Tools menu

Discussion Questions

- What is different between a normal push-button switch and a capacitive touch sensor?

- Why does the code need to remember the previous touch state (`lastTouchState`) instead of just checking the current state?
- What might happen if we removed the delay after detecting a touch? Why might that cause problems?
- Can you think of a real-world device you have seen or used that uses capacitive touch sensing?
- Why do you think our bodies are able to trigger the sensor just by getting close, without needing to press hard like a button?

Extension Challenges

- **Easy:** Change the code so touching the sensor makes the LED blink three times quickly instead of just turning on or off.
- **Medium:** Add a second LED and modify the code so each touch cycles through three states: both LEDs off, only LED 1 on, both LEDs on.
- **Hard:** Use the touch sensor to control the brightness of an LED with PWM (`analogWrite`), so each touch increases brightness in steps, cycling back to off after the maximum brightness is reached.

Parent Guidance Tips

- Let your child lead — step in only when they're stuck for more than a minute or two.
- Ask "what do you think will happen?" before each step to build prediction skills.
- Mistakes are part of it — a project that doesn't work first time is a learning moment, not a failure.
- Take a short break between projects to keep energy up.
- Celebrate the finished build — take a photo and talk about what they'd change next time.